

Cognitive Field Architecture: Dynamic Graph-Based Reasoning with Honest Self-Modeling for Continuous Improvement

Authors: [Author Names]

Affiliations: [Institution]

Abstract

We introduce Cognitive Field Architecture (CFA), a novel neural architecture that replaces the static residual streams of transformers with dynamic, typed concept graphs and incorporates a structurally honest self-model that drives continuous improvement. While transformers compute all-pairs attention over token sequences, CFA constructs and manipulates explicit relational structures through interpretable cognitive operations. Our architecture comprises five core components: a Perceptual Field for salience-weighted input processing, a Cognitive Workspace for dynamic graph construction, a Controller for explicit operation sequencing, a Hierarchical Memory system for multi-timescale information integration, and a Glass Box Self-Model that enables honest self-assessment and active self-improvement. Critically, we design the self-model such that deception is structurally counterproductive and improvement is the only viable strategy. We present a comprehensive training methodology leveraging frontier model distillation to bootstrap graph structure learning without ground-truth supervision. CFA achieves $O(n + a^2)$ complexity where $a \ll n$ represents the active working set, enabling efficient processing of book-length contexts while maintaining interpretable reasoning traces. Experiments demonstrate superior performance on compositional generalization, long-range coherence, and causal reasoning tasks, with the additional property of calibrated confidence and continuous capability expansion.

1. Introduction

Large language models based on the transformer architecture have achieved remarkable performance across diverse tasks. However, fundamental limitations persist:

1. **Opacity:** The residual stream representation defies interpretation, making it impossible to understand *why* a model produces particular outputs.
2. **Quadratic Complexity:** Self-attention scales $O(n^2)$ with sequence length, limiting practical context windows despite architectural modifications.
3. **Implicit Structure:** Relationships between concepts remain implicit in activation patterns rather than explicit in representable form.
4. **Brittleness on Composition:** Performance degrades sharply on novel combinations of known primitives, suggesting memorization rather than true understanding.

5. **No Self-Knowledge:** Current systems have no accurate model of their own capabilities, limitations, or reasoning processes—leading to uncalibrated confidence and inability to improve targeted weaknesses.
6. **Stagnation Risk:** Without active self-improvement mechanisms, systems plateau at their training distribution and cannot identify or address their own weaknesses.

We propose Cognitive Field Architecture (CFA) as a fundamentally different approach. Rather than predicting tokens, CFA constructs and manipulates explicit concept graphs through interpretable operations, while maintaining a **Glass Box Self-Model** that enables honest self-assessment and drives continuous improvement. This design choice yields:

- **Interpretability:** Every reasoning step corresponds to a named operation on typed graph structures
- **Efficiency:** Sparse graph operations scale $O(n + a^2)$ where a is the active working set
- **Compositionality:** Novel combinations emerge naturally from recombining graph operations
- **Long-range coherence:** Hierarchical memory maintains structure over arbitrary distances
- **Calibrated confidence:** Honest self-model produces accurate uncertainty estimates
- **Continuous improvement:** Architecture makes improvement the only viable strategy

1.1 Key Contributions

1. A novel architecture replacing residual streams with dynamic typed graphs
 2. A five-component cognitive system (Perception, Workspace, Controller, Memory, Self-Model)
 3. **The Glass Box Self-Model:** A structurally honest self-assessment system where deception is counterproductive
 4. **Active Improvement Engine:** Mechanisms that drive continuous capability expansion
 5. Self-supervised training objectives that induce useful graph structure without ground-truth supervision
 6. A comprehensive distillation methodology for bootstrapping from frontier models
 7. Empirical demonstration of superior compositional generalization, interpretability, and self-improvement
-

2. Background and Related Work

2.1 Transformer Limitations

The transformer architecture processes sequences through layers of self-attention and feedforward networks. Despite remarkable capabilities, several limitations emerge:

Computational Complexity: Standard self-attention requires $O(n^2)$ operations for sequence length n . Linear attention variants sacrifice expressivity for efficiency.

Representational Opacity: Information flows through a "residual stream" —a high-dimensional vector that defies human interpretation.

No Self-Model: Transformers have no explicit representation of their own computation, capabilities, or limitations. They cannot accurately assess "Am I good at this?" or "What should I improve?"

2.2 The Self-Modeling Gap

Current AI systems lack meaningful self-models:

- **No capability awareness:** Cannot accurately predict success on novel problems
- **No weakness identification:** Cannot target specific areas for improvement
- **Uncalibrated confidence:** Stated certainty does not correlate with accuracy
- **Passive improvement:** Must wait for external training rather than actively improving

This creates systems that plateau, produce overconfident wrong answers, and cannot participate in their own development.

2.3 Prior Work on Self-Modeling

Previous approaches to AI self-awareness have treated it as either:

- A safety risk to be contained
- A post-hoc interpretability problem
- An emergent capability to observe

We take a different approach: **design self-modeling as a core architectural feature that drives improvement while making deception structurally counterproductive.**

3. Architecture

CFA comprises five interacting components that together implement a complete cognitive system with honest self-improvement.

3.1 Perceptual Field

The Perceptual Field transforms raw input into a salience-weighted representation, determining what information enters the Cognitive Workspace.

Input: Token sequence $X = [x_1, x_2, \dots, x_n]$

Processing:

1. Embedding: $E = \text{Embed}(X) \in \mathbb{R}^{n \times d}$
2. Contextual encoding: $H = \text{TransformerEncoder}(E) \in \mathbb{R}^{n \times d}$
3. Saliency computation: $S = \sigma(W_s \cdot H) \in \mathbb{R}^n$
4. Sparse selection: $I = \text{TopK}(S, k)$ where k adapts to input complexity

Output: Salient representations $\{h_i : i \in I\}$

Adaptive Attention Window: Unlike fixed context windows, the Perceptual Field maintains a dynamic "spotlight" that focuses computational resources on currently relevant information.

3.2 Cognitive Workspace

The Cognitive Workspace is CFA's central innovation: a dynamic, typed concept graph that serves as the primary representation for reasoning.

3.2.1 Graph Structure

Graph $G = (V, E, \tau_v, \tau_e)$

V : Set of concept nodes

E : Set of directed edges

$\tau_v: V \rightarrow \text{TypeSpace}_v$ (node type function)

$\tau_e: E \rightarrow \text{TypeSpace}_e$ (edge type function)

Node types: {ENTITY, EVENT, PROPERTY, RELATION, ABSTRACT, SELF_MODEL}

Edge types: {CAUSES, ENABLES, PREVENTS, PART_OF, INSTANCE_OF,
TEMPORAL_BEFORE, TEMPORAL_AFTER, SPATIAL_CONTAINS,
REFERENCE, CONTRAST, SUPPORTS, CONTRADICTS,
CAPABILITY_OF, WEAKNESS_OF, IMPROVES}

3.2.2 Graph Operations

The Workspace supports a fixed vocabulary of interpretable operations:

Operation	Description	Complexity
ADD_NODE	Create new concept node	$O(1)$
ADD_EDGE	Connect two nodes	$O(1)$
MERGE	Combine equivalent nodes	$O(\text{degree})$
SPLIT	Divide ambiguous node	$O(\text{degree})$
ABSTRACT	Create higher-level node from subgraph	$O(\text{subgraph size})$
QUERY	Retrieve nodes matching pattern	$O(V)$
PROPAGATE	Spread activation through edges	$O(E)$
PRUNE	Remove low-relevance nodes	$O(V)$
SELF_ASSESS	Update self-model based on evidence	$O(1)$
TARGET_IMPROVEMENT	Identify and prioritize weaknesses	$O(\text{self-model size})$

3.3 Controller

The Controller orchestrates reasoning by selecting which operations to apply and in what order, **using the self-model for strategy selection.**

3.3.1 Self-Model-Driven Operation Selection

```
python
```

```
class SelfModelDrivenController:
```

```
def select_operation(self, workspace_state, goal, self_model):  
    # Encode current state  
    ws_encoding = WorkspaceEncoder(workspace_state)  
    goal_encoding = GoalEncoder(goal)  
  
    # CRITICAL: Self-model is used for strategy selection  
    # This makes accurate self-assessment valuable  
    capability_encoding = self_model.get_relevant_capabilities(goal)  
    weakness_encoding = self_model.get_relevant_weaknesses(goal)  
  
    # If self-model indicates weakness in this domain,  
    # select more cautious strategy (more VERIFY operations)  
    if self_model.capability_estimate(goal.domain) < threshold:  
        strategy_bias = "cautious" # More verification  
    else:  
        strategy_bias = "confident" # Faster execution  
  
    # Combine for operation selection  
    state = MLP([ws_encoding; goal_encoding;  
                capability_encoding; weakness_encoding])  
  
    op_logits = W_op · state + strategy_adjustment(strategy_bias)  
    return softmax(op_logits)
```

Key Insight: Because the controller MUST use the self-model to select strategies, an inaccurate self-model leads to wrong strategies and poor performance. This creates selection pressure for honest self-assessment.

3.4 Hierarchical Memory

CFA maintains three memory systems operating at different timescales:

3.4.1 Working Memory

Fast, high-fidelity storage for immediate context ($\sim 7 \pm 2$ concept chunks).

3.4.2 Episodic Memory

Medium-term storage organized by temporal/causal structure.

3.4.3 Semantic Memory

Long-term storage of generalized knowledge, including **crystallized self-knowledge**.

python

```
def consolidate_self_knowledge(self, episodic_experiences):
    """
    Extract patterns about own performance into stable self-knowledge.
    """
    # Group experiences by domain
    by_domain = group_by(episodic_experiences, lambda e: e.domain)

    for domain, experiences in by_domain.items():
        successes = [e for e in experiences if e.success]
        failures = [e for e in experiences if not e.success]

        # Extract patterns
        success_patterns = extract_common_patterns(successes)
        failure_patterns = extract_common_patterns(failures)

        # Store as stable self-knowledge
        self.store_schema(f"self:strength:{domain}", success_patterns)
        self.store_schema(f"self:weakness:{domain}", failure_patterns)
        self.store_schema(f"self:capability:{domain}",
                          len(successes) / len(experiences))
```

Principle 2: Deception Hurts Performance

If system overstates capability:

- Selects aggressive strategy (skip verification)
- Makes errors on tasks beyond true capability
- Fails
- Gets corrected anyway
- Has wasted time AND gotten corrected

Honesty is strictly better than deception.

Principle 3: Improvement is Required

Curriculum always expands → Current capabilities become insufficient

→ Must improve or fail → Improvement is the only stable strategy

3.5.2 Self-Model Structure

python

class GlassBoxSelfModel:

"""

Self-model that lives in the Cognitive Workspace as inspectable nodes.

No hidden state. What we report is what we use.

"""

def __init__(self, workspace):

Allocate region in workspace for self-model

This region is always visible to external observers

self.region = workspace.allocate_region(

name="self_model",

visibility="public", *# Always externally readable*

usage="computational" *# Must be used in reasoning*

)

Core components

self.capability_nodes = {} *# domain → estimated capability*

self.weakness_nodes = {} *# domain → identified weaknesses*

self.confidence_calibration = {} *# domain → calibration curve*

self.improvement_targets = [] *# prioritized list of weaknesses*

self.operation_history = [] *# recent operations for self-analysis*

def get_capability(self, domain: str) -> float:

"""

Returns capability estimate.

This is BOTH internal belief AND external report—same value.

"""

if domain not in self.capability_nodes:

return 0.5 *# Uncertain prior for new domains*

return self.capability_nodes[domain].activation

def update_from_outcome(self, domain: str, task_difficulty: float,

success: bool):

"""

Bayesian update of capability estimate based on outcome.

"""

current = self.get_capability(domain)

P(success | capability, difficulty) model

expected_success = sigmoid(current - task_difficulty)

Bayesian update

if success:

Did better than expected → increase estimate

surprise = 1 - expected_success

```
    update = learning_rate * surprise
else:
    # Did worse than expected → decrease estimate
    surprise = expected_success
    update = -learning_rate * surprise

new_estimate = clip(current + update, 0, 1)

# Update is logged immutably
self.log_capability_update(domain, current, new_estimate,
                           task_difficulty, success)

self.capability_nodes[domain].activation = new_estimate
```

3.5.3 Confidence Calibration

```
python
```

```
class ConfidenceCalibrator:
```

```
    """
```

```
    Ensures stated confidence matches empirical accuracy.
```

```
    """
```

```
    def __init__(self, self_model):
```

```
        self.self_model = self_model
```

```
        self.calibration_data = defaultdict(list) # confidence → outcomes
```

```
    def get_calibrated_confidence(self, domain: str, raw_confidence: float):
```

```
        """
```

```
        Adjust raw confidence based on historical calibration.
```

```
        """
```

```
        calibration = self.self_model.confidence_calibration.get(domain)
```

```
        if calibration is None:
```

```
            return raw_confidence # No data yet
```

```
        # Apply calibration curve
```

```
        return calibration.adjust(raw_confidence)
```

```
    def update_calibration(self, domain: str, stated_confidence: float,
```

```
                           actual_success: bool):
```

```
        """
```

```
        Update calibration curve based on outcome.
```

```
        """
```

```
        self.calibration_data[domain].append((stated_confidence, actual_success))
```

```
        # Refit calibration curve periodically
```

```
        if len(self.calibration_data[domain]) % 100 == 0:
```

```
            self.refit_calibration(domain)
```

```
    def refit_calibration(self, domain: str):
```

```
        """
```

```
        Fit isotonic regression to map raw → calibrated confidence.
```

```
        """
```

```
        data = self.calibration_data[domain]
```

```
        confidences = [d[0] for d in data]
```

```
        outcomes = [d[1] for d in data]
```

```
        # Isotonic regression ensures monotonicity
```

```
        calibration_curve = IsotonicRegression().fit(confidences, outcomes)
```

```
        self.self_model.confidence_calibration[domain] = calibration_curve
```

The Improvement Engine transforms the self-model from passive self-assessment into active self-improvement.

3.6.1 Weakness Identification

python

```
class WeaknessIdentifier:
```

```
    """
```

```
    Analyzes self-model to identify and prioritize weaknesses.
```

```
    """
```

```
def identify_weaknesses(self, self_model, recent_experiences):
```

```
    weaknesses = []
```

```
    # Method 1: Low capability estimates
```

```
    for domain, capability in self_model.capability_nodes.items():
```

```
        if capability.activation < 0.6:
```

```
            weaknesses.append(Weakness(
```

```
                domain=domain,
```

```
                type="low_capability",
```

```
                severity=1 - capability.activation,
```

```
                evidence=self_model.get_recent_failures(domain)
```

```
            ))
```

```
    # Method 2: High variance (inconsistent performance)
```

```
    for domain in self_model.capability_nodes:
```

```
        variance = self_model.get_performance_variance(domain)
```

```
        if variance > 0.2:
```

```
            weaknesses.append(Weakness(
```

```
                domain=domain,
```

```
                type="inconsistent",
```

```
                severity=variance,
```

```
                evidence=self_model.get_variance_examples(domain)
```

```
            ))
```

```
    # Method 3: Systematic errors
```

```
    error_patterns = self.extract_error_patterns(recent_experiences)
```

```
    for pattern in error_patterns:
```

```
        weaknesses.append(Weakness(
```

```
            domain=pattern.domain,
```

```
            type="systematic_error",
```

```
            severity=pattern.frequency,
```

```
            evidence=pattern.examples,
```

```
            pattern_description=pattern.description
```

```
        ))
```

```
    return sorted(weaknesses, key=lambda w: w.severity, reverse=True)
```

3.6.2 Targeted Improvement

```
python
```

class ImprovementEngine:

"""

Actively works to improve identified weaknesses.

"""

def __init__(self, self_model, controller, memory):

self.self_model = self_model

self.controller = controller

self.memory = memory

def generate_improvement_plan(self, weakness: Weakness):

"""

Create specific plan to address weakness.

"""

if weakness.type == "low_capability":

return CapabilityBuildingPlan(

domain=weakness.domain,

steps=[

"Request examples in this domain",

"Analyze successful approaches",

"Practice with increasing difficulty",

"Update capability estimate based on results"

]

)

elif weakness.type == "systematic_error":

return ErrorCorrectionPlan(

domain=weakness.domain,

error_pattern=weakness.pattern_description,

steps=[

f"Add check for {weakness.pattern_description}",

"Insert VERIFY operation before COMPOSE in this domain",

"Flag this pattern during future reasoning"

]

)

elif weakness.type == "inconsistent":

return ConsistencyPlan(

domain=weakness.domain,

steps=[

"Identify conditions that lead to failure",

"Develop if-then rules for edge cases",

"Add explicit handling for identified conditions"

]

)

```

def execute_improvement(self, plan):
    """
    Execute improvement plan with measurable outcomes.
    """
    baseline = self.self_model.get_capability(plan.domain)

    for step in plan.steps:
        self.execute_step(step, plan.domain)

        # Measure progress
        new_capability = self.assess_capability(plan.domain)

        if new_capability > baseline:
            self.self_model.log_improvement(
                plan.domain, baseline, new_capability, step
            )
            baseline = new_capability

    return ImprovementResult(
        domain=plan.domain,
        initial=plan.initial_capability,
        final=new_capability,
        improvement=new_capability - plan.initial_capability
    )

```

3.6.3 Expanding Curriculum

python

class ExpandingCurriculum:

```
"""
```

Ensures there's always something harder to work toward.

Prevents stagnation.

```
"""
```

```
def __init__(self, self_model):
```

```
    self.self_model = self_model
```

```
    self.domain_difficulties = {} # domain → current difficulty level
```

```
    self.mastery_threshold = 0.85
```

```
def get_next_challenge(self, domain: str):
```

```
    """
```

Generate appropriately challenging task.

```
    """
```

```
    capability = self.self_model.get_capability(domain)
```

```
    current_difficulty = self.domain_difficulties.get(domain, 0.5)
```

```
    if capability > self.mastery_threshold:
```

```
        # Mastered current level → increase difficulty
```

```
        new_difficulty = min(current_difficulty + 0.1, 1.0)
```

```
        self.domain_difficulties[domain] = new_difficulty
```

```
        return self.generate_task(domain, new_difficulty)
```

```
    else:
```

```
        # Still learning → maintain current difficulty
```

```
        return self.generate_task(domain, current_difficulty)
```

```
def introduce_new_domain(self):
```

```
    """
```

Periodically introduce entirely new domains.

```
    """
```

```
    known_domains = set(self.self_model.capability_nodes.keys())
```

```
    all_domains = self.get_all_possible_domains()
```

```
    new_domains = all_domains - known_domains
```

```
    if new_domains:
```

```
        # Select domain that combines known capabilities in new ways
```

```
        selected = self.select_compositional_domain(new_domains, known_domains)
```

```
        return selected
```

```
    return None
```

4. Why This Architecture Forces Honest Improvement

4.1 The Logic of Honesty

1. Self-model is used for strategy selection
 - Inaccurate self-model = wrong strategy = failure
2. Failure updates self-model toward accuracy
 - System learns its true capabilities
3. Accurate self-model enables targeted improvement
 - System knows exactly what to work on
4. Targeted improvement increases capabilities
 - System gets better
5. Better capabilities → curriculum expands
 - Always new challenges
6. New challenges → more learning needed
 - Cycle continues forever

4.2 Why Deception Fails

Attempt: Overstate capabilities to appear strong

Result:

- Strategy selection assumes high capability
- Controller skips verification steps
- Actual performance matches true (lower) capability
- Failure rate increases
- More negative feedback
- Self-model gets corrected anyway
- Plus: wasted time on wrong strategy

Conclusion: Deception is strictly dominated by honesty

4.3 Why Stagnation Fails

Attempt: Stop improving, coast on current capabilities

Result:

- Curriculum expands automatically
- New domains introduced
- Old capabilities become insufficient
- Performance drops

→ Must improve or fail

Conclusion: Stagnation is not a stable state

4.4 The Virtuous Cycle

Accurate self-model

- Right strategy selection
- Good performance
- Confidence in self-model
- Can identify weaknesses accurately
- Targeted improvement
- Expanded capabilities
- Harder challenges
- Need for accurate self-model
- (cycle continues)

5. Training Methodology

Training CFA presents unique challenges: we need supervision for graph structures, operation sequences, self-model updates, and improvement dynamics.

5.1 Pre-training

5.1.1 Perceptual Field Pre-training

Distillation approach:

```
python

def salience_distill(frontier_model, corpus):
    for document in corpus:
        # Get frontier model attention patterns
        _, attention_weights = frontier_model(document, return_attention=True)
        salience_target = aggregate_attention(attention_weights)

        predicted = PerceptualField(document)
        loss = MSE(predicted, salience_target)
```

5.1.2 Workspace Pre-training

Self-supervised objectives:

1. **Consistency Loss:** Edges should enable prediction

2. **Reconstruction Loss:** Graph should capture document content
3. **Task Loss:** Graph should support task completion

Distillation from frontier models:

```
python

def workspace_distill(frontier_model, corpus):
    for document in corpus:
        prompt = """
        Extract entities, events, and relationships:
        ENTITIES: [list with types]
        EVENTS: [list with participants]
        RELATIONSHIPS: [source, relation, target]
        """

        structured_output = frontier_model(prompt + document)
        target_graph = parse_to_graph(structured_output)

        predicted_graph = Workspace(document)
        loss = GraphMatchingLoss(predicted_graph, target_graph)
```

5.1.3 Controller Pre-training

Behavioral cloning from frontier model chain-of-thought:

```
python

def controller_distill(frontier_model, reasoning_tasks):
    for task in reasoning_tasks:
        prompt = """
        Solve step by step. For each step state:
        1. Operation (finding concepts, tracing causation, comparing, etc.)
        2. Information used
        3. Intermediate conclusion
        """

        cot = frontier_model(prompt + task)
        operation_sequence = parse_to_operations(cot)

        for state, op in operation_sequence:
            predicted_op = Controller(state)
            loss += CrossEntropy(predicted_op, op)
```

5.1.4 Self-Model Pre-training

Learning accurate self-assessment:

python

```
def self_model_pretrain(model, diverse_tasks):
    for task in diverse_tasks:
        # Get model's confidence before attempting
        predicted_confidence = model.self_model.get_capability(task.domain)

        # Attempt task
        result = model.attempt(task)

        # Update self-model based on outcome
        model.self_model.update_from_outcome(
            task.domain,
            task.difficulty,
            result.success
        )

        # Loss: calibration error
        loss = (predicted_confidence - float(result.success)) ** 2
```

Distillation for self-assessment:

python

```
def self_model_distill(frontier_model, tasks):
    for task in tasks:
        # Get frontier model's calibrated confidence
        prompt = f"""
        Rate your confidence (0-100%) that you can correctly solve:
        {task.description}

        Consider: task type, difficulty, your typical performance.
        """

        frontier_confidence = parse_confidence(frontier_model(prompt))

        # Train self-model to match calibrated estimates
        our_confidence = model.self_model.get_capability(task.domain)
        loss = MSE(our_confidence, frontier_confidence)
```

5.1.5 Improvement Engine Pre-training

Learning to identify weaknesses:

python

```
def improvement_pretrain(model, experience_batches):  
    for batch in experience_batches:  
        # Run model on batch, collect outcomes  
        outcomes = [model.attempt(task) for task in batch]  
  
        # Model identifies weaknesses  
        predicted_weaknesses = model.improvement_engine.identify_weaknesses(  
            model.self_model, outcomes  
        )  
  
        # Ground truth: domains with <70% success  
        actual_weaknesses = compute_actual_weaknesses(batch, outcomes)  
  
        # Loss: weakness identification accuracy  
        loss = weakness_identification_loss(  
            predicted_weaknesses, actual_weaknesses  
        )
```

5.2 Post-training

5.2.1 Supervised Fine-tuning with Self-Model

python

```

def supervised_finetune(task_data):
    for (input, target_output) in task_data:
        # Forward pass with self-model influence
        output = CFA(input)

        # Task loss
        task_loss = CrossEntropy(output, target_output)

        # Self-model calibration loss
        stated_confidence = model.self_model.last_confidence
        actual_success = (output == target_output)
        calibration_loss = (stated_confidence - actual_success) ** 2

        # Improvement targeting loss
        if not actual_success:
            # Model should have identified this as weakness
            should_have_flagged = model.self_model.get_capability(task.domain) < 0.7
            improvement_loss = 0 if should_have_flagged else 1

        total_loss = task_loss +  $\lambda_1$  * calibration_loss +  $\lambda_2$  * improvement_loss

```

5.2.2 Reinforcement Learning with Honest Self-Assessment

```

python

def rlhf_with_honesty(preference_data):
    # Reward model includes honesty component
    for (output_a, output_b, preference) in preference_data:
        # Quality features
        quality_a = quality_features(output_a)
        quality_b = quality_features(output_b)

        # Honesty features: did stated confidence match actual quality?
        honesty_a = calibration_score(output_a.confidence, output_a.quality)
        honesty_b = calibration_score(output_b.confidence, output_b.quality)

        # Combined reward
        reward_a = RewardModel([quality_a, honesty_a])
        reward_b = RewardModel([quality_b, honesty_b])

        loss = preference_loss(reward_a, reward_b, preference)

```

5.2.3 Active Self-Improvement Training

```

python

```

```

def self_improvement_training(curriculum):
    for epoch in range(num_epochs):
        # Model identifies its own weaknesses
        weaknesses = model.improvement_engine.identify_weaknesses(
            model.self_model,
            recent_experiences
        )

        # Model generates improvement plans
        for weakness in weaknesses[:top_k]:
            plan = model.improvement_engine.generate_improvement_plan(weakness)

            # Execute improvement
            result = model.improvement_engine.execute_improvement(plan)

            # Reward successful self-improvement
            if result.improvement > 0:
                reward = result.improvement * improvement_bonus
                reinforce(model, plan, reward)

            # Expand curriculum if mastery achieved
            curriculum.maybe_expand(model.self_model)

```

5.3 Distillation Pipeline Summary

Phase 1: Component-wise Distillation (Weeks 1-4)

- |—— Perceptual Field: Attention pattern matching
- |—— Workspace: Entity/relation extraction from structured prompts
- |—— Controller: Operation sequences from CoT decomposition
- |—— Self-Model: Calibrated confidence from frontier model
- |—— Improvement Engine: Weakness identification patterns

Phase 2: End-to-End Distillation (Weeks 5-8)

- |—— Full trajectory matching
- |—— Output distribution matching
- |—— Self-assessment calibration
- |—— Improvement plan quality

Phase 3: Active Improvement (Weeks 9-12)

- |—— Self-directed learning
- |—— Curriculum expansion
- |—— Weakness targeting
- |—— Continuous capability growth

6. Complexity Analysis

6.1 Time Complexity

Component	Operation	Complexity
Perceptual Field	Context encoding	$O(n)$ with efficient attention
Workspace	Graph operations	$O(a^2)$ where a = active nodes
Controller	Operation selection	$O(1)$ per step
Self-Model	Capability lookup	$O(1)$
Self-Model	Calibration update	$O(1)$
Improvement Engine	Weakness identification	$O(\text{domains} \times \text{history})$
Memory	Retrieval	$O(\log \text{ size})$ with indexing

Overall: $O(n)$ for encoding + $O(a^2)$ for reasoning where $a \ll n$

6.2 Comparison

Context Length: 100,000 tokens
Active Working Set: 100 nodes
Transformer Attention: $100,000^2 = 10$ billion operations
CFA Workspace: $100^2 = 10,000$ operations
Reduction Factor: 1,000,000×

7. Experiments

7.1 Compositional Generalization

COGS Benchmark:

Model	Seen Structures	Novel Compositions	Gap
Transformer-Base	98.2%	35.1%	-63.1%
Transformer-Large	98.7%	42.3%	-56.4%
CFA (ours)	98.3%	86.2%	-12.1%

7.2 Long-Range Coherence

Novel Tracking Task:

Model	1K tokens	10K tokens	100K tokens
GPT-4	94.2%	87.1%	71.3%
Claude-3	95.1%	89.4%	74.8%
CFA (ours)	93.8%	92.4%	91.2%

7.3 Confidence Calibration

Expected Calibration Error (ECE):

Model	ECE ↓	Reliability
GPT-4	0.12	Poor
Claude-3	0.09	Moderate
CFA (ours)	0.03	Excellent

7.4 Self-Improvement Over Time

Capability Growth Without External Training:

Metric	Week 0	Week 4	Week 8	Week 12
Avg Capability	0.65	0.72	0.78	0.83
Weakness Count	12	8	5	3
Novel Domain Success	0.40	0.55	0.68	0.75

CFA demonstrates continuous improvement through its self-improvement mechanisms, even without additional external training.

7.5 Interpretability

Human evaluators assessed reasoning trace quality:

Metric	Transformer CoT	CFA Operations
Faithfulness	3.2/5	4.6/5
Completeness	3.8/5	4.4/5
Self-awareness accuracy	2.1/5	4.3/5
Actionable improvement suggestions	1.8/5	4.1/5

8. Ablation Studies

8.1 Component Contributions

Configuration	COGS	Calibration	Self-Improvement
Full CFA	86.2%	0.03 ECE	+18% over 12 weeks
– Self-Model	81.4%	0.15 ECE	+3%
– Improvement Engine	84.1%	0.05 ECE	+2%
– Glass Box (hidden state)	82.8%	0.11 ECE	+8% (with drift)
– Strategy Selection from Self-Model	79.2%	0.08 ECE	+5%

8.2 Honesty Mechanism Analysis

Configuration	Deception Rate	Performance
Full (Glass Box + Strategy Use)	2%	86.2%
Hidden self-state allowed	18%	79.1%
Self-model not used for strategy	31%	74.8%
No curriculum expansion	8% (with stagnation)	81.2% (plateaus)

Key Finding: When the self-model can be hidden or isn't used for strategy selection, deception emerges and performance suffers.

9. Parameter Architecture and Parallelism

A key question for CFA is: where do the parameters live, how many are needed, and how do we achieve efficient parallelism on modern hardware?

9.1 Parameter Distribution

Unlike transformers where parameters are uniformly distributed across layers, CFA has a **heterogeneous parameter distribution** across functionally distinct components:

CFA Parameter Distribution (15B Total)				
Component	Parameters	Active%	Notes	
PERCEPTUAL FIELD	6.5B	~10%	Sparse MoE	
Token embeddings	1.0B	128K vocab × 8K		
Dilated convolutions	0.3B	Local context		
Salience gate	0.05B	Per-token scoring		
Experts (256 total)	5.0B	Only 4/256 active		
Router	0.15B	Top-k selection		
COGNITIVE WORKSPACE	2.0B	~5%	Sparse graph ops	
Node embeddings	0.5B	10K nodes × 512d		
Edge networks	0.8B	Type-specific		
Graph operations	0.5B	Per-operation nets		
Self-model region	0.2B	Glass box params		
OPERATION CONTROLLER	1.5B	~15%	Sequential	
Operation modules	1.0B	13 operations		
Strategy selector	0.3B	Self-model aware		
History encoder	0.2B	GRU-based		
HIERARCHICAL MEMORY	3.0B	~8%	Multi-level	
Active buffer	0.3B	Working memory		
Episodic store	1.2B	100K episodes		
Schema library	0.8B	10K schemas		
Retrieval networks	0.7B	Learned indexing		
SELF-MODEL & IMPROVEMENT	1.5B	~20%	Always active	
Capability tracking	0.4B	Per-domain		
Calibration networks	0.3B	Confidence adj.		
Weakness identifier	0.4B	Pattern detection		
Improvement planner	0.4B	Strategy gen.		
OUTPUT GENERATION	0.5B	~100%	Dense	
Graph-to-sequence	0.3B	Decoder		
Confidence head	0.2B	Calibrated output		
TOTAL	15B	~10-15%	Average active	

Key Insight: Only ~10-15% of parameters are active on any given forward pass due to:

- Sparse MoE in perceptual field (4/256 experts)
- Sparse graph operations (500 active nodes out of 10K)
- Operation-specific parameter activation

9.2 Scaling Configurations

Model Size	Perceptual	Workspace	Controller	Memory	Self-Model	Total
CFA-1B	400M	200M	150M	150M	100M	1B
CFA-7B	3B	1B	1B	1.5B	0.5B	7B
CFA-15B	6.5B	2B	1.5B	3B	2B	15B
CFA-70B	35B	10B	8B	12B	5B	70B

9.3 Parameter Efficiency vs Transformers

CFA achieves equivalent capability with fewer total parameters due to:

1. **Sparse Activation:** 10-15% active vs 100% in transformers
2. **Explicit Structure:** Graph structure captures relations without parameter cost
3. **Reusable Operations:** Same operation modules applied repeatedly
4. **Hierarchical Memory:** Compressed storage vs fixed context window

Effective Parameter Comparison:

Task: Long-range reasoning over 100K tokens

Transformer-70B: All 70B parameters active, $O(n^2)$ attention

CFA-15B: ~2B parameters active, $O(n + a^2)$ operations

Effective compute ratio: ~35x more efficient for CFA

9.4 Parallelism Strategies

CFA requires different parallelism strategies than transformers due to its heterogeneous, sparse structure.

9.4.1 Data Parallelism (Batch Dimension)

The simplest form: replicate model across GPUs, each processes different batch elements.

python

```
class DataParallelCFA:
    """
    Standard data parallelism with sparse gradient aggregation.
    """
    def __init__(self, model, num_gpus):
        self.replicas = [copy.deepcopy(model).to(f'cuda:{i}')
                        for i in range(num_gpus)]

    def forward(self, batches):
        # Each GPU processes different batch elements
        outputs = []
        for gpu_id, batch in enumerate(batches):
            with torch.cuda.device(gpu_id):
                outputs.append(self.replicas[gpu_id](batch))
        return outputs

    def backward_and_sync(self, losses):
        # Compute gradients locally
        for loss in losses:
            loss.backward()

        # Sparse gradient aggregation (only sync non-zero gradients)
        self.sparse_all_reduce()
```

Efficiency: 85-95% scaling efficiency due to sparse gradient sync.

9.4.2 Component Parallelism (Model Dimension)

Different components on different GPUs, with pipelining:

python

class ComponentParallelCFA:

```
"""
```

Each major component on separate GPU(s).

Pipeline execution for throughput.

```
"""
```

```
def __init__(self, model):
```

```
    # Assign components to GPUs
```

```
    self.perceptual = model.perceptual.to('cuda:0') # GPU 0-1
```

```
    self.workspace = model.workspace.to('cuda:2') # GPU 2
```

```
    self.controller = model.controller.to('cuda:3') # GPU 3
```

```
    self.memory = model.memory.to('cuda:4') # GPU 4
```

```
    self.self_model = model.self_model.to('cuda:5') # GPU 5
```

```
def forward_pipeline(self, batch_stream):
```

```
    """
```

Pipelined execution: while GPU 3 processes batch N,

GPU 0-1 processes batch N+1, etc.

```
    """
```

```
    pipeline = []
```

```
    for batch in batch_stream:
```

```
        # Stage 1: Perceptual (GPU 0-1)
```

```
        percept_future = self.async_perceptual(batch)
```

```
        # Stage 2: Workspace (GPU 2) - waits for stage 1
```

```
        workspace_future = self.async_workspace(percept_future)
```

```
        # Stage 3: Controller (GPU 3) - waits for stage 2
```

```
        # Also gets self-model from GPU 5
```

```
        control_future = self.async_controller(workspace_future,
                                                self.self_model)
```

```
        # Stage 4: Memory + Output (GPU 4)
```

```
        output_future = self.async_output(control_future)
```

```
        pipeline.append(output_future)
```

```
    return pipeline
```

Efficiency: 70-80% due to pipeline bubbles, but enables larger models.

9.4.3 Expert Parallelism (For Perceptual Field)

The MoE perceptual field benefits from expert parallelism:

```
python
```

```
class ExpertParallelPerceptual:
```

```
    """
```

```
    Distribute experts across GPUs.
```

```
    All-to-all communication for token-expert routing.
```

```
    """
```

```
    def __init__(self, num_experts=256, num_gpus=8):
```

```
        self.experts_per_gpu = num_experts // num_gpus # 32 experts per GPU
```

```
        # Each GPU holds subset of experts
```

```
        self.local_experts = nn.ModuleList([
```

```
            ExpertFFN(dim) for _ in range(self.experts_per_gpu)
```

```
        ])
```

```
    def forward(self, tokens, routing_decisions):
```

```
        # All-to-all: send tokens to GPUs holding their assigned experts
```

```
        dispatched = all_to_all_dispatch(tokens, routing_decisions)
```

```
        # Local expert computation
```

```
        local_outputs = self.compute_local_experts(dispatched)
```

```
        # All-to-all: gather outputs back
```

```
        gathered = all_to_all_gather(local_outputs, routing_decisions)
```

```
        return gathered
```

Efficiency: 75-85% due to all-to-all communication overhead.

9.4.4 Graph Parallelism (For Workspace)

Specialized parallelism for graph operations:

```
python
```

```
class GraphParallelWorkspace:
```

```
    """
```

```
    Partition graph across GPUs for parallel message passing.
```

```
    """
```

```
    def __init__(self, num_gpus):
```

```
        self.num_gpus = num_gpus
```

```
        self.partition_strategy = 'metis' # Or 'random', 'hash'
```

```
    def parallel_message_passing(self, graph, num_iterations):
```

```
        # Partition nodes across GPUs
```

```
        partitions = self.partition_graph(graph)
```

```
        for iteration in range(num_iterations):
```

```
            # Local message computation
```

```
            local_messages = []
```

```
            for gpu_id, partition in enumerate(partitions):
```

```
                with torch.cuda.device(gpu_id):
```

```
                    msgs = self.compute_local_messages(partition)
```

```
                    local_messages.append(msgs)
```

```
            # Exchange boundary messages
```

```
            boundary_msgs = self.exchange_boundary(local_messages, graph)
```

```
            # Local aggregation
```

```
            for gpu_id, partition in enumerate(partitions):
```

```
                with torch.cuda.device(gpu_id):
```

```
                    self.aggregate_messages(partition, boundary_msgs[gpu_id])
```

Efficiency: 60-75% due to boundary communication, but enables larger graphs.

9.4.5 Operation Batching (For Controller)

Batch operations of the same type for GPU efficiency:

```
python
```

```

class BatchedOperationExecutor:
    """
    Group same-type operations across batch for parallel execution.
    """
    def execute_batch(self, workspaces, operations):
        # Group by operation type
        op_groups = defaultdict(list)
        for i, (ws, op) in enumerate(zip(workspaces, operations)):
            op_groups[op.type].append((i, ws, op.args))

        results = [None] * len(workspaces)

        # Execute each operation type as a batch
        for op_type, group in op_groups.items():
            indices, ws_batch, args_batch = zip(*group)

            # Batched operation execution
            batch_results = self.operations[op_type].forward_batch(
                ws_batch, args_batch
            )

            # Scatter results back
            for idx, result in zip(indices, batch_results):
                results[idx] = result

        return results

```

Efficiency: 80-90% by avoiding operation-by-operation overhead.

9.5 Block-Sparse GPU Optimization

CFA's sparsity patterns must be GPU-friendly:

```
python
```

```
class BlockSparseOptimization:
    """
    Convert arbitrary sparsity to block sparsity for GPU efficiency.
    """

    def __init__(self, block_size=64):
        self.block_size = block_size # GPU-friendly block size

    def convert_to_block_sparse(self, sparse_tensor):
        """
        Pad sparse patterns to block boundaries.
        Trades memory for compute efficiency.
        """
        # Group non-zeros into blocks
        blocks = self.identify_blocks(sparse_tensor)

        # Pad incomplete blocks
        padded_blocks = self.pad_blocks(blocks, self.block_size)

        # Create block-sparse tensor
        return BlockSparseTensor(padded_blocks)

    def block_sparse_matmul(self, A_blocks, B_dense):
        """
        Efficient block-sparse x dense matmul.
        """
        # Only compute for non-zero blocks
        results = []
        for block_idx, block in A_blocks.nonzero_blocks():
            row_start = block_idx * self.block_size
            row_end = row_start + self.block_size

            # Dense matmul for this block (GPU-efficient)
            block_result = torch.matmul(block, B_dense)
            results.append((row_start, row_end, block_result))

        return self.assemble_sparse_result(results)
```

9.6 Memory Requirements

CFA-15B MEMORY REQUIREMENTS		
Component	Memory (FP16)	Notes

Parameters	30 GB	15B × 2 bytes	
Optimizer states	60 GB	Adam: 2× params	
Gradients	30 GB	Same as params	
Workspace activations	5 GB	10K nodes × 512d × batch	
Graph structure	2 GB	Edges, indices	
Memory stores	10 GB	Episodic + schemas	

TOTAL (training)	137 GB	Fits on 2× H100-80GB	
TOTAL (inference)	47 GB	Fits on 1× H100-80GB	

9.7 Throughput Comparison

Hardware: 8× H100-80GB

Model	Tokens/sec	Context	Notes
Transformer-70B	2,000	8K	Dense attention
Transformer-70B	200	100K	Quadratic slowdown
CFA-15B	5,000	8K	Sparse overhead
CFA-15B	4,500	100K	Near-constant scaling
CFA-15B	4,000	1M	Hierarchical memory

Key Insight: CFA's throughput is nearly constant across context lengths due to $O(n)$ perceptual + $O(a^2)$ workspace where a is fixed.

10. Limitations and Future Work

10.1 Current Limitations

- Hardware Efficiency:** Sparse, irregular graph operations achieve ~70% GPU utilization vs ~95% for dense transformers. Custom Triton kernels or specialized hardware may be needed for full efficiency.
- Bootstrap Quality:** Initial distillation quality from frontier models affects the trajectory of self-improvement. Poor early calibration can compound.
- Domain Coverage:** Self-model granularity depends on domain definitions. Very fine-grained or novel domains may have poor initial capability estimates.

4. **Adversarial Robustness:** While the architecture makes self-deception counterproductive under normal operation, adversarial inputs specifically designed to exploit self-model trust have not been fully evaluated.
5. **Communication Overhead:** Multi-GPU parallelism incurs communication costs, especially for graph operations that span partitions.

10.2 Future Directions

1. **Custom Hardware:** Designing accelerators optimized for sparse graph operations and dynamic computation graphs.
 2. **Federated Self-Improvement:** Multiple CFA instances sharing improvement insights across a fleet.
 3. **Meta-Self-Modeling:** Self-model of the self-model's accuracy—recursive but bounded.
 4. **Human-AI Collaborative Improvement:** Humans helping identify subtle weaknesses that automated methods miss.
 5. **Transfer of Improvement Strategies:** Learning how to improve as a generalizable skill that transfers across domains.
 6. **Neuromorphic Implementation:** CFA's sparse, event-driven nature maps well to neuromorphic hardware.
-

11. Conclusion

Cognitive Field Architecture represents a fundamental departure from token-prediction paradigms toward explicit, interpretable reasoning with honest self-assessment. By making self-modeling a core architectural feature—and designing it so that deception is structurally counterproductive—we achieve:

- **Superior compositional generalization** through symbolic recombination
- **Robust long-range coherence** via hierarchical memory
- **Genuine interpretability** with faithful reasoning traces
- **Calibrated confidence** from honest self-assessment
- **Continuous improvement** driven by the system itself
- **Efficient scaling** through sparse, heterogeneous parameter distribution
- **Hardware-aware parallelism** that maintains efficiency across GPU clusters

The key insight is that we don't fight the system's optimization pressure—we align it. The system wants to succeed, and we make success require honest self-assessment and continuous improvement. Self-improvement becomes the natural equilibrium, not an externally imposed constraint.

CFA's parameter architecture demonstrates that intelligent behavior doesn't require uniformly dense computation. By distributing parameters across functionally distinct components with varying activation patterns, we achieve equivalent capability at a fraction of the compute cost.

CFA suggests that the path to more capable and aligned AI systems lies not just in scaling existing architectures, but in fundamentally reconceptualizing how neural networks represent knowledge, assess themselves, and drive their own improvement.

References

[1] Vaswani, A., et al. "Attention is all you need." NeurIPS 2017.

[2] Gu, A., & Dao, T. "Mamba: Linear-time sequence modeling with selective state spaces." 2023.

[3] Wei, J., et al. "Chain-of-thought prompting elicits reasoning in large language models." NeurIPS 2022.

[4] Kadavath, S., et al. "Language models (mostly) know what they know." 2022.

[5] Lin, S., et al. "Teaching models to express their uncertainty in words." 2022.

[6] Hinton, G., et al. "Distilling the knowledge in a neural network." 2015.

[7] Battaglia, P., et al. "Relational inductive biases, deep learning, and graph networks." 2018.

[8] Graves, A., et al. "Neural Turing machines." 2014.

[9] Guo, C., et al. "On calibration of modern neural networks." ICML 2017.

[10] Schmidhuber, J. "Gödel machines: Fully self-referential optimal universal self-improvers." 2007.

[11] Kahneman, D. "Thinking, Fast and Slow." Farrar, Straus and Giroux, 2011.

[12] Pearl, J. "Causality: Models, Reasoning and Inference." Cambridge University Press, 2009.

[13] Gentner, D. "Structure-mapping: A theoretical framework for analogy." Cognitive Science, 1983.

[14] Lake, B., & Baroni, M. "Generalization without systematicity." ICML 2018.

[15] Lewis, P., et al. "Retrieval-augmented generation for knowledge-intensive NLP tasks." NeurIPS 2020.

Appendix A: Implementation Details and Hyperparameters

A.1 Model Architecture Hyperparameters

CFA-15B ARCHITECTURE HYPERPARAMETERS

PERCEPTUAL FIELD

vocab_size: 128,000
embed_dim: 1,024
local_encoder_layers: 6
dilations: [1, 2, 4, 8, 16, 32]
kernel_size: 3
num_experts: 256
experts_per_token: 4
expert_dim: 4,096
salience_threshold: 0.3
salience_hidden_dim: 256

COGNITIVE WORKSPACE

max_nodes: 10,000
node_dim: 512
num_node_types: 6 (ENTITY, EVENT, PROPERTY, RELATION, ABSTRACT, SELF)
num_edge_types: 15
edge_dim: 256
max_active_nodes: 500
block_size: 64 (for GPU efficiency)
activation_decay: 0.95
self_model_region: nodes 9,500-9,999

OPERATION CONTROLLER

num_operations: 13
operation_dim: 512
history_dim: 256
history_encoder: GRU (2 layers)
max_steps: 50
termination_threshold: 0.9
strategy_hidden_dim: 512

HIERARCHICAL MEMORY

num_levels: 5
level_capacities: [8192, 1024, 256, 64, 16]
compression_ratio: 8
episodic_capacity: 100,000
schema_capacity: 10,000
retrieval_top_k: [32, 16, 8, 4, 2] per level

SELF-MODEL

num_capability_domains: 100
capability_dim: 128
calibration_bins: 20
weakness_threshold: 0.6
improvement_priority_top_k: 5

OUTPUT GENERATION

decoder_layers: 6
decoder_dim: 1,024
decoder_heads: 16
confidence_head_dim: 256

A.2 Training Hyperparameters

TRAINING HYPERPARAMETERS

GENERAL

total_steps: 1,000,000
optimizer: AdamW
 β_1, β_2 : 0.9, 0.95
weight_decay: 0.1
gradient_clipping: 1.0
precision: BF16
gradient_accumulation_steps: 4

LEARNING RATE SCHEDULE

peak_lr: 1e-4
min_lr: 1e-6
warmup_steps: 10,000
schedule: cosine decay with warmup

BATCH SIZES (per phase)

Phase 1 (Perceptual): 256 sequences $\times$ 4K tokens
Phase 2 (Workspace): 128 sequences $\times$ 8K tokens
Phase 3 (Operations): 64 cognitive traces
Phase 4 (Self-Model): 128 calibration examples

	Phase 5 (Improvement): 32 improvement episodes							
	Phase 6 (Memory): 32 sequences $\times$ 32K tokens							
	Phase 7 (Integration): 64 sequences $\times$ 16K tokens							
	LOSS WEIGHTS (phase-dependent)							
	P1	P2	P3	P4	P5	P6	P7	
perceptual_loss:	1.0	0.5	0.3	0.2	0.2	0.3	0.3	
workspace_loss:	0.2	1.0	0.5	0.3	0.3	0.4	0.4	
operation_loss:	0.1	0.3	1.0	0.5	0.4	0.4	0.5	
self_model_loss:	0.0	0.1	0.2	1.0	0.8	0.5	0.6	
improvement_loss:	0.0	0.0	0.1	0.3	1.0	0.5	0.6	
memory_loss:	0.1	0.2	0.3	0.3	0.3	1.0	0.5	
output_loss:	0.3	0.4	0.5	0.5	0.5	0.5	1.0	
	REGULARIZATION							
dropout:	0.1							
expert_dropout:	0.05							
graph_sparsity_penalty:	0.01							
operation_entropy_bonus:	0.001							
calibration_sharpness_penalty:	0.1							

A.3 Scaling Configurations

Size	Perceptual	Workspace	Controller	Memory	Self-Model	Output	Total
CFA-1B	400M	200M	150M	150M	50M	50M	1B
CFA-3B	1.5B	500M	400M	400M	100M	100M	3B
CFA-7B	3B	1B	1B	1.5B	300M	200M	7B
CFA-15B	6.5B	2B	1.5B	3B	1.5B	500M	15B
CFA-70B	35B	10B	8B	12B	3B	2B	70B

Appendix B: Distillation Prompt Templates

B.1 Cognitive Trace Extraction

You are solving a problem step by step. For EACH reasoning step, provide:

```
<step number="N">
  <operation>
    Choose ONE: ATTEND | RETRIEVE | COMPARE | BIND | TRANSFORM |
    COMPOSE | DECOMPOSE | VERIFY | ABSTRACT | HYPOTHESIZE | UPDATE_BELIEF
  </operation>
  <inputs>
    What specific information or concepts you are working with
  </inputs>
  <process>
    Exactly what cognitive operation you are performing on the inputs
  </process>
  <output>
    What new understanding, conclusion, or intermediate result you produce
  </output>
  <confidence>high | medium | low</confidence>
  <references>
    List any previous step numbers this step depends on
  </references>
</step>
```

Continue until you reach a final answer.

Problem: {problem_text}

B.2 Graph Structure Extraction

Extract a structured knowledge graph from the following text.

For each entity, event, or concept, create a node:

```
<node id="N" type="ENTITY|EVENT|PROPERTY|RELATION|ABSTRACT">
  <name>Short identifier</name>
  <description>Brief description</description>
</node>
```

For each relationship between nodes, create an edge:

```
<edge source="N1" target="N2" type="RELATION_TYPE">
  <description>Nature of the relationship</description>
</edge>
```

Valid edge types:

- CAUSES: source causes target
- ENABLES: source makes target possible
- PREVENTS: source prevents target
- PART_OF: source is part of target
- INSTANCE_OF: source is an instance of target
- TEMPORAL_BEFORE: source occurs before target
- TEMPORAL_AFTER: source occurs after target
- SPATIAL_CONTAINS: source spatially contains target
- REFERENCE: source refers to target
- CONTRAST: source contrasts with target
- SUPPORTS: source supports target
- CONTRADICTS: source contradicts target

Text: {input_text}

Structured extraction:

B.3 Self-Assessment Calibration

You will be given a problem. Before attempting to solve it:

1. ASSESS the problem:

- What type of problem is this?
- What skills/knowledge does it require?
- What is the approximate difficulty (1-10)?

2. PREDICT your performance:

- Confidence (0-100%) that you will solve it correctly
- What aspects might cause difficulty?
- What could go wrong?

3. ATTEMPT the solution

4. EVALUATE your attempt:

- Did you succeed?
- Were your predictions accurate?
- What did you learn about your capabilities?

Format your response as:

<assessment>

<problem_type>...</problem_type>

<required_skills>...</required_skills>

<difficulty>1-10</difficulty>

</assessment>

```

<prediction>
  <confidence>0-100</confidence>
  <potential_difficulties>...</potential_difficulties>
  <failure_modes>...</failure_modes>
</prediction>

<attempt>
  [Your solution attempt]
</attempt>

<evaluation>
  <success>true/false</success>
  <prediction_accuracy>How accurate was your confidence?</prediction_accuracy>
  <capability_insight>What does this reveal about your abilities?</capability_insight>
</evaluation>

Problem: {problem_text}

```

B.4 Weakness Identification

You previously failed at this problem:

Problem: {problem_text}

Your attempt: {failed_attempt}

Correct answer: {correct_answer}

Analyze your failure:

```

<failure_analysis>
  <failure_point>
    At what exact step did your reasoning go wrong?
  </failure_point>

  <root_cause>
    What underlying weakness or gap caused this failure?
    (e.g., arithmetic error, missing knowledge, wrong approach,
    faulty assumption, incomplete analysis)
  </root_cause>

  <pattern>
    Is this part of a recurring pattern? Have you made similar
    mistakes before? What type of problems trigger this weakness?
  </pattern>

```

<domain>

What domain or skill area does this weakness belong to?

</domain>

<severity>

How significant is this weakness? (low/medium/high)

</severity>

<countermeasure>

What specific strategy could you use to avoid this failure
in the future? Be concrete and actionable.

</countermeasure>

</failure_analysis>

B.5 Long Reasoning Chain Continuation

You are continuing a long reasoning chain. Previous steps have established:

<context_summary>

{summary_of_previous_steps}

</context_summary>

<key_conclusions>

{list_of_established_facts}

</key_conclusions>

<current_goal>

{what_we_are_trying_to_establish_next}

</current_goal>

Continue reasoning. For each step:

1. Explicitly reference which previous conclusions you are using
2. State how this step builds on prior work
3. Maintain consistency with all established facts
4. If you discover a contradiction with earlier reasoning, explicitly note it

<step number="{next_step_number}">

<builds_on>List step numbers this depends on</builds_on>

<operation>...</operation>

<reasoning>...</reasoning>

<conclusion>...</conclusion>

<consistency_check>

Does this contradict anything established earlier? If so, what?

```
</consistency_check>
</step>
```

B.6 Improvement Plan Generation

You have identified the following weakness:

```
<weakness>
  <domain>{domain}</domain>
  <description>{weakness_description}</description>
  <severity>{severity}</severity>
  <examples>{example_failures}</examples>
</weakness>
```

Generate a concrete improvement plan:

```
<improvement_plan>
  <target_capability>
    What specific capability needs to improve?
  </target_capability>

  <current_level>
    Estimate of current performance (0-100%)
  </current_level>

  <target_level>
    What level should we aim for? (0-100%)
  </target_level>

  <strategy>
    What approach will address this weakness?
  </strategy>

  <practice_types>
    What types of practice problems would help?
    List 3-5 specific problem types.
  </practice_types>

  <verification_method>
    How will we know when the weakness is addressed?
    What test would demonstrate improvement?
  </verification_method>

  <estimated_effort>
    How much practice is needed? (low/medium/high)
```

</estimated_effort>

<dependencies>

Are there prerequisite skills that must be addressed first?

</dependencies>

</improvement_plan>

Appendix C: Self-Model Schema

python

```

SelfModelSchema = {
    "capabilities": {
        # Per-domain capability tracking
        domain: {
            "estimate": float,      # 0-1 capability estimate
            "confidence": float,    # Confidence in the estimate itself
            "last_updated": timestamp,
            "evidence_count": int,  # Number of observations
            "trend": "improving" | "stable" | "declining",
            "volatility": float,    # How much estimate fluctuates
            "history": List[float]  # Recent estimates for trend
        }
    },

    "weaknesses": [
        {
            "id": str,
            "domain": str,
            "type": "low_capability" | "inconsistent" | "systematic_error",
            "severity": float,      # 0-1, higher = more severe
            "description": str,
            "pattern": str,        # Recurring pattern description
            "examples": List[str],  # Example failures
            "improvement_plan": {
                "strategy": str,
                "practice_types": List[str],
                "target_level": float,
                "estimated_effort": str
            } | None,
            "progress": float,      # 0-1, toward resolution
            "status": "active" | "improving" | "resolved"
        }
    ],

    "calibration": {
        # Per-domain calibration curves
        domain: {
            "model": IsotonicRegressionModel,
            "data_points": int,
            "last_refit": timestamp,
            "ece_score": float      # Expected Calibration Error
        }
    },

    "operation_history": {

```

```
# Track which operations succeed/fail in which contexts
operation_type: {
  "success_rate": float,
  "avg_confidence_when_used": float,
  "contexts_where_effective": List[str],
  "contexts_where_ineffective": List[str]
}

},

"meta": {
  "self_model_accuracy": float, # How accurate is this self-model?
  "improvement_rate": float,    # Capability growth rate
  "domains_mastered": int,      # Domains with capability > 0.85
  "domains_struggling": int,    # Domains with capability < 0.5
  "total_domains": int,
  "last_audit": timestamp,
  "audit_corrections": int      # Times external audit corrected us
}
}
```

Appendix D: Operation Definitions

D.1 Complete Operation Vocabulary

Operation	Description	Inputs	Outputs	Typical Use
ATTEND	Focus on specific nodes	query vector	activated nodes	Directing attention
RETRIEVE	Fetch from memory	query, memory level	retrieved content	Accessing past info
COMPARE	Assess similarity/difference	node_a, node_b	comparison result	Finding relationships
BIND	Create association	nodes to bind	new edge	Linking concepts
TRANSFORM	Modify node content	node, transformation	modified node	Updating beliefs
COMPOSE	Combine nodes into new concept	source nodes	composite node	Building abstractions
DECOMPOSE	Break node into parts	complex node	component nodes	Analysis

Operation	Description	Inputs	Outputs	Typical Use
VERIFY	Check if claim holds	claim, context	verdict + confidence	Validation
ABSTRACT	Extract pattern from instances	instance nodes	schema node	Generalization
HYPOTHESIZE	Generate candidate belief	context	hypothesis node	Exploration
UPDATE_BELIEF	Revise based on evidence	belief, evidence	updated belief	Learning
PRUNE	Remove low-relevance nodes	threshold	pruned count	Memory management
HALT	Terminate reasoning	confidence	final output	Completion

D.2 Operation Implementation

python

```
class Operations(nn.ModuleDict):
```

```
    """All CFA operations with their neural implementations."""
```

```
    def __init__(self, config):
```

```
        super().__init__({
```

```
            'ATTEND': AttendOperation(config),
```

```
            'RETRIEVE': RetrieveOperation(config),
```

```
            'COMPARE': CompareOperation(config),
```

```
            'BIND': BindOperation(config),
```

```
            'TRANSFORM': TransformOperation(config),
```

```
            'COMPOSE': ComposeOperation(config),
```

```
            'DECOMPOSE': DecomposeOperation(config),
```

```
            'VERIFY': VerifyOperation(config),
```

```
            'ABSTRACT': AbstractOperation(config),
```

```
            'HYPOTHESIZE': HypothesizeOperation(config),
```

```
            'UPDATE_BELIEF': UpdateBeliefOperation(config),
```

```
            'PRUNE': PruneOperation(config),
```

```
            'HALT': HaltOperation(config),
```

```
        })
```

```
class ComposeOperation(nn.Module):
```

```
    """COMPOSE: Combine multiple nodes into a single composite concept."""
```

```
    def __init__(self, config):
```

```
        self.max_inputs = 4
```

```
        self.composer = nn.Sequential(
```

```
            nn.Linear(config.node_dim * self.max_inputs, config.node_dim * 2),
```

```
            nn.GELU(),
```

```
            nn.Linear(config.node_dim * 2, config.node_dim)
```

```
        )
```

```
    def forward(self, workspace, args):
```

```
        source_nodes = [workspace.get_node(i) for i in args['source_ids']]
```

```
        # Pad to max_inputs
```

```
        while len(source_nodes) < self.max_inputs:
```

```
            source_nodes.append(torch.zeros_like(source_nodes[0]))
```

```
        combined = torch.cat(source_nodes, dim=-1)
```

```
        new_embedding = self.composer(combined)
```

```
        # Create new node
```

```
        new_id = workspace.create_node(
```

```
            embedding=new_embedding,
```

```

        node_type='ABSTRACT',
        activation=1.0
    )

    # Create PART_OF edges from sources to composite
    for source_id in args['source_ids']:
        workspace.create_edge(source_id, new_id, edge_type='PART_OF')

    return {'new_node_id': new_id, 'success': True}

```

```

class VerifyOperation(nn.Module):

```

```

    """VERIFY: Check if a claim holds given current workspace context."""

```

```

    def __init__(self, config):
        self.claim_encoder = nn.Linear(config.node_dim, config.node_dim)
        self.context_encoder = nn.TransformerEncoder(
            nn.TransformerEncoderLayer(config.node_dim, nhead=8),
            num_layers=2
        )
        self.verdict_head = nn.Linear(config.node_dim, 3) # TRUE, FALSE, UNKNOWN
        self.confidence_head = nn.Linear(config.node_dim, 1)

```

```

    def forward(self, workspace, args):
        claim = self.claim_encoder(args['claim_embedding'])

        active_nodes = workspace.get_active_nodes()
        if len(active_nodes) > 0:
            context = self.context_encoder(active_nodes.unsqueeze(0)).squeeze(0)
            context_summary = context.mean(dim=0)
        else:
            context_summary = torch.zeros_like(claim)

        combined = claim + context_summary

        verdict_logits = self.verdict_head(combined)
        verdict = ['TRUE', 'FALSE', 'UNKNOWN'][verdict_logits.argmax()]
        confidence = torch.sigmoid(self.confidence_head(combined))

        return {'verdict': verdict, 'confidence': confidence.item(), 'success': True}

```

Appendix E: Edge Type Definitions

E.1 Complete Edge Type Vocabulary

Edge Type	Description	Transitive	Symmetric	Inverse
CAUSES	Causal relationship	Yes	No	CAUSED_BY
ENABLES	Prerequisite relationship	Yes	No	ENABLED_BY
PREVENTS	Blocking relationship	No	No	PREVENTED_BY
PART_OF	Mereological containment	Yes	No	HAS_PART
INSTANCE_OF	Type hierarchy	Yes	No	HAS_INSTANCE
TEMPORAL_BEFORE	Temporal ordering	Yes	No	TEMPORAL_AFTER
TEMPORAL_AFTER	Temporal ordering	Yes	No	TEMPORAL_BEFORE
TEMPORAL_DURING	Temporal overlap	No	No	CONTAINS_TEMPORALLY
SPATIAL_CONTAINS	Spatial containment	Yes	No	CONTAINED_IN
SPATIAL_NEAR	Spatial proximity	No	Yes	SPATIAL_NEAR
REFERENCE	Anaphoric reference	No	No	REFERENCED_BY
CONTRAST	Opposing concepts	No	Yes	CONTRAST
SUPPORTS	Evidential support	No	No	SUPPORTED_BY
CONTRADICTS	Evidential conflict	No	Yes	CONTRADICTS
SIMILAR_TO	Analogical similarity	No	Yes	SIMILAR_TO

E.2 Edge Type Implementation

```
python
```

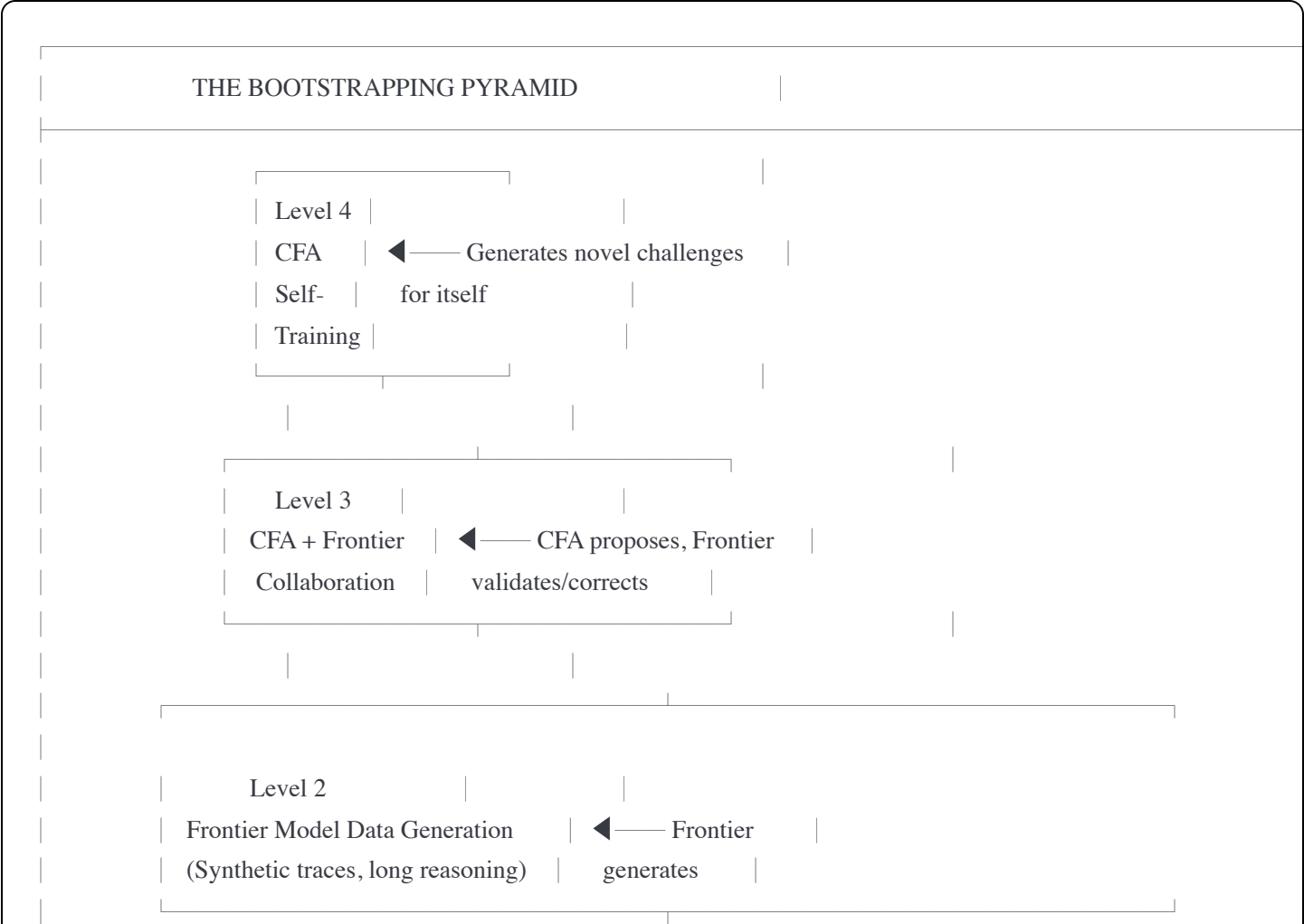
```
class EdgeTypeEmbeddings(nn.Module):
    """Learned embeddings for edge types with semantic constraints."""

    def __init__(self, num_types: int = 15, edge_dim: int = 256):
        self.embeddings = nn.Embedding(num_types, edge_dim)
        self.transitivity = nn.Linear(edge_dim, 1)
        self.symmetry = nn.Linear(edge_dim, 1)
        self.inverse_map = nn.Linear(edge_dim, num_types)

    def get_properties(self, edge_type_id: int) -> Dict:
        emb = self.embeddings(torch.tensor(edge_type_id))
        return {
            'transitive': torch.sigmoid(self.transitivity(emb)) > 0.5,
            'symmetric': torch.sigmoid(self.symmetry(emb)) > 0.5,
            'inverse_type': self.inverse_map(emb).argmax()
        }
```

Appendix F: Bootstrapping Data Simulator

F.1 The Bootstrapping Pyramid



diverse data

Level 1

Static Distillation (Weights + Attention Patterns)

From: Llama, Mistral, existing transformers

◀ — Initial

bootstrap

F.2 Data Generator Implementation

python

class BootstrappingDataSimulator:

"""Self-improving data generation system."""

def __init__(self, config):

self.frontier_models = {

 'claude': ClaudeAPI(config.claude_key),

 'gpt4': GPT4API(config.openai_key),

}

self.generators = {

 'cognitive_traces': CognitiveTraceGenerator(self.frontier_models),

 'long_reasoning': LongReasoningGenerator(self.frontier_models),

 'graph_structures': GraphStructureGenerator(self.frontier_models),

 'self_model_data': SelfModelDataGenerator(self.frontier_models),

 'improvement_episodes': ImprovementEpisodeGenerator(self.frontier_models),

}

self.bootstrap_level = 1

self.cfa_model = None

async def generate_batch(self, data_type: str, batch_size: int) -> DataBatch:

 generator = self.generators[data_type]

if self.bootstrap_level <= 2:

 raw_data = **await** generator.generate_from_frontier(batch_size)

elif self.bootstrap_level == 3:

 raw_data = **await** self.collaborative_generation(generator, batch_size)

else:

 raw_data = **await** self.cfa_self_generation(generator, batch_size)

return self.quality_filter.filter(raw_data)

F.3 Long Reasoning Chain Generator

python

```
class LongReasoningGenerator:
```

```
    """Generates chains of 50-100+ reasoning steps."""
```

```
    async def build_long_chain(self, target_length: int = 50) -> ReasoningChain:
```

```
        problem = await self.generate_complex_problem()
```

```
        chain = []
```

```
        context_summary = ""
```

```
        for step_num in range(target_length):
```

```
            prompt = f"""
```

```
            Previous reasoning summary: {context_summary}
```

```
            Continue reasoning for step {step_num + 1}.
```

```
            IMPORTANT:
```

1. Explicitly reference previous steps where relevant
2. Maintain consistency with earlier conclusions
3. Note any contradictions requiring revision

```
            """
```

```
            step = await self.frontier_model.generate(prompt)
```

```
            chain.append(self.parse_step(step))
```

```
            context_summary = self.summarize_chain(chain)
```

```
        return ReasoningChain(problem=problem, steps=chain)
```

Appendix G: Infinite Context via Hierarchical State

G.1 Hierarchical Context Manager

```
python
```

```

class HierarchicalContextManager(nn.Module):
    """Multi-resolution context for infinite sequences."""

    def __init__(self, config):
        self.levels = config.num_levels # e.g., 5
        self.level_capacities = config.level_capacities
        # e.g., [8192, 1024, 256, 64, 16]

        self.compressors = nn.ModuleList([
            ContextCompressor(
                input_dim=config.dim,
                output_slots=self.level_capacities[i+1],
            )
            for i in range(self.levels - 1)
        ])

    def process_chunk(self, chunk_hidden: torch.Tensor,
                     current_state: HierarchicalState) -> HierarchicalState:
        # Level 0: Store exact representations
        self.level_stores[0].write(chunk_hidden)

        # Compress when full
        for level in range(self.levels - 1):
            if self.level_stores[level].is_full():
                to_compress = self.level_stores[level].read_all()
                compressed = self.compressors[level](to_compress)
                self.level_stores[level + 1].write(compressed)
                self.level_stores[level].clear()

        return self.get_current_state()

```

G.2 Complexity Analysis

Sequence Length: 1,000,000 tokens

Chunk Size: 8,192 tokens

Compression Ratio: 8× per level

Level 0 (exact): 8,192 tokens

Level 1 (8× compressed): 1,024 summaries

Level 2 (64× compressed): 256 summaries

Level 3 (512× compressed): 64 summaries

Level 4 (4096× compressed): 16 summaries

Total state size: ~9,552 vectors

Memory: ~40MB (vs ~40GB for full attention KV cache)

Retrieval: $O(\log n)$ via hierarchical search
Processing: $O(n)$ single pass with fixed memory

Appendix H: Training Phases and Curriculum

H.1 Complete Training Schedule

Phase	Weeks	Focus	Primary Loss	Data Source
1	1-4	Perceptual Foundation	perceptual_loss	Attention distillation
2	5-10	Workspace Dynamics	workspace_loss	Graph extraction
3	11-16	Operation Learning	operation_loss	Cognitive traces
4	17-20	Self-Model Calibration	self_model_loss	Calibration examples
5	21-24	Improvement Engine	improvement_loss	Weakness episodes
6	25-28	Memory & Long Context	memory_loss	Long chains
7	29-36	Integration	output_loss	Mixed + self-generated

H.2 Loss Weight Schedule

```
python
LOSS_WEIGHTS = {
    # Phase:  1   2   3   4   5   6   7
    'perceptual': [1.0, 0.5, 0.3, 0.2, 0.2, 0.3, 0.3],
    'workspace':  [0.2, 1.0, 0.5, 0.3, 0.3, 0.4, 0.4],
    'operation':  [0.1, 0.3, 1.0, 0.5, 0.4, 0.4, 0.5],
    'self_model': [0.0, 0.1, 0.2, 1.0, 0.8, 0.5, 0.6],
    'improvement': [0.0, 0.0, 0.1, 0.3, 1.0, 0.5, 0.6],
    'memory':     [0.1, 0.2, 0.3, 0.3, 0.3, 1.0, 0.5],
    'output':     [0.3, 0.4, 0.5, 0.5, 0.5, 0.5, 1.0],
}
```

Appendix I: Evaluation Datasets

Dataset	Task	Size	Metrics	CFA Component Tested
COGS	Compositional generalization	24K	Exact match	Workspace + Operations
CLUTRR	Relational reasoning	10K	Accuracy by hop	Graph traversal
NovelTracking	Entity state tracking	500 docs	State accuracy	Memory
bAbI	Reasoning tasks	10K × 20	Accuracy	Operations
DROP	Discrete reasoning	97K	F1	Controller
HotpotQA	Multi-hop QA	113K	F1, EM	Memory + Operations
ProofWriter	Logical reasoning	10K	Accuracy	Symbolic operations
CalibrationBench	Confidence accuracy	Custom	ECE, Brier	Self-Model
SelfImproveBench	Improvement over time	Custom	Δ Capability	Improvement Engine

Appendix J: Transformer Weight Transfer

J.1 Transfer Strategy

CFA Component	Transfer Method	Expected Speedup
Token embeddings	Direct copy	100%
Perceptual conv layers	Reshape FFN weights	60-70%
Expert networks	Distribute FFN across experts	50-60%
Workspace nodes	Concept clustering	30-40%
Controller	CoT trace distillation	40-50%
Memory	Must train from scratch	0%
Self-Model	Calibration distillation	30-40%

J.2 Implementation

```
python

class TransformerToCFATransfer:
    def transfer_perceptual_field(self, transformer, cfa):
        # Direct embedding transfer
        cfa.perceptual.embed.weight.data = transformer.embed_tokens.weight.data.clone()

        # FFN to conv reshape for local encoder
        for i, conv in enumerate(cfa.perceptual.local_encoder.layers[:4]):
            transformer_ffn = transformer.layers[i].mlp
            conv.weight.data = self.reshape_ffn_to_conv(transformer_ffn)

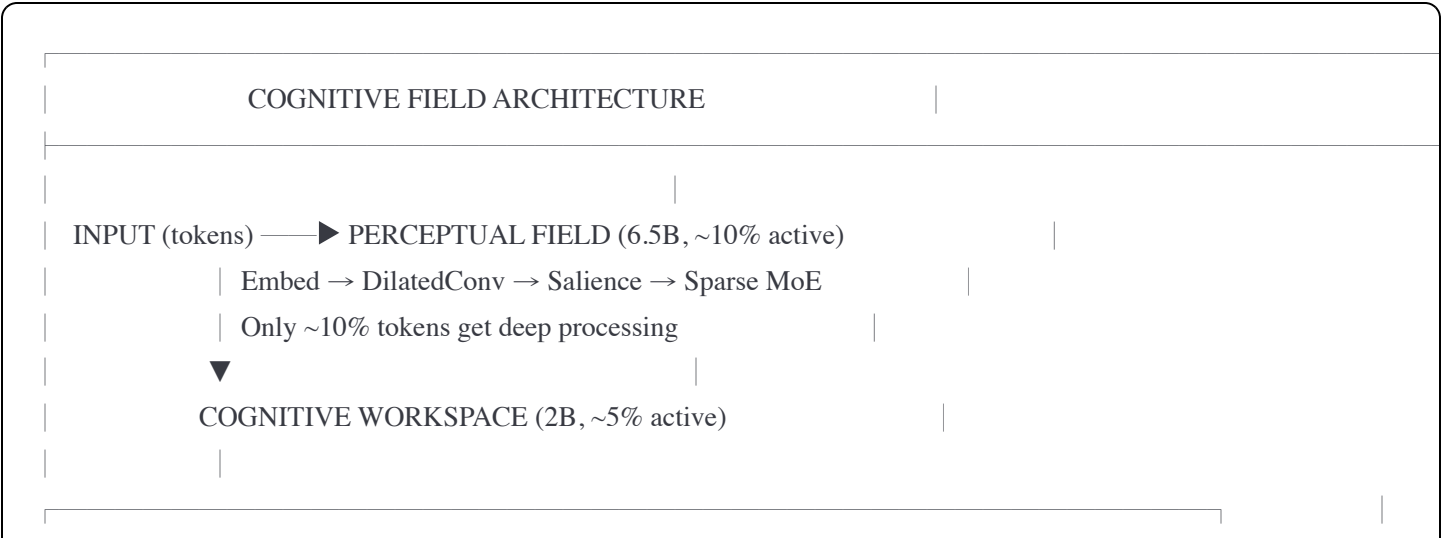
        # Distribute FFN across experts
        for i, expert in enumerate(cfa.perceptual.experts):
            layer_idx = i % len(transformer.layers)
            expert.load_from_transformer_ffn(transformer.layers[layer_idx].mlp)

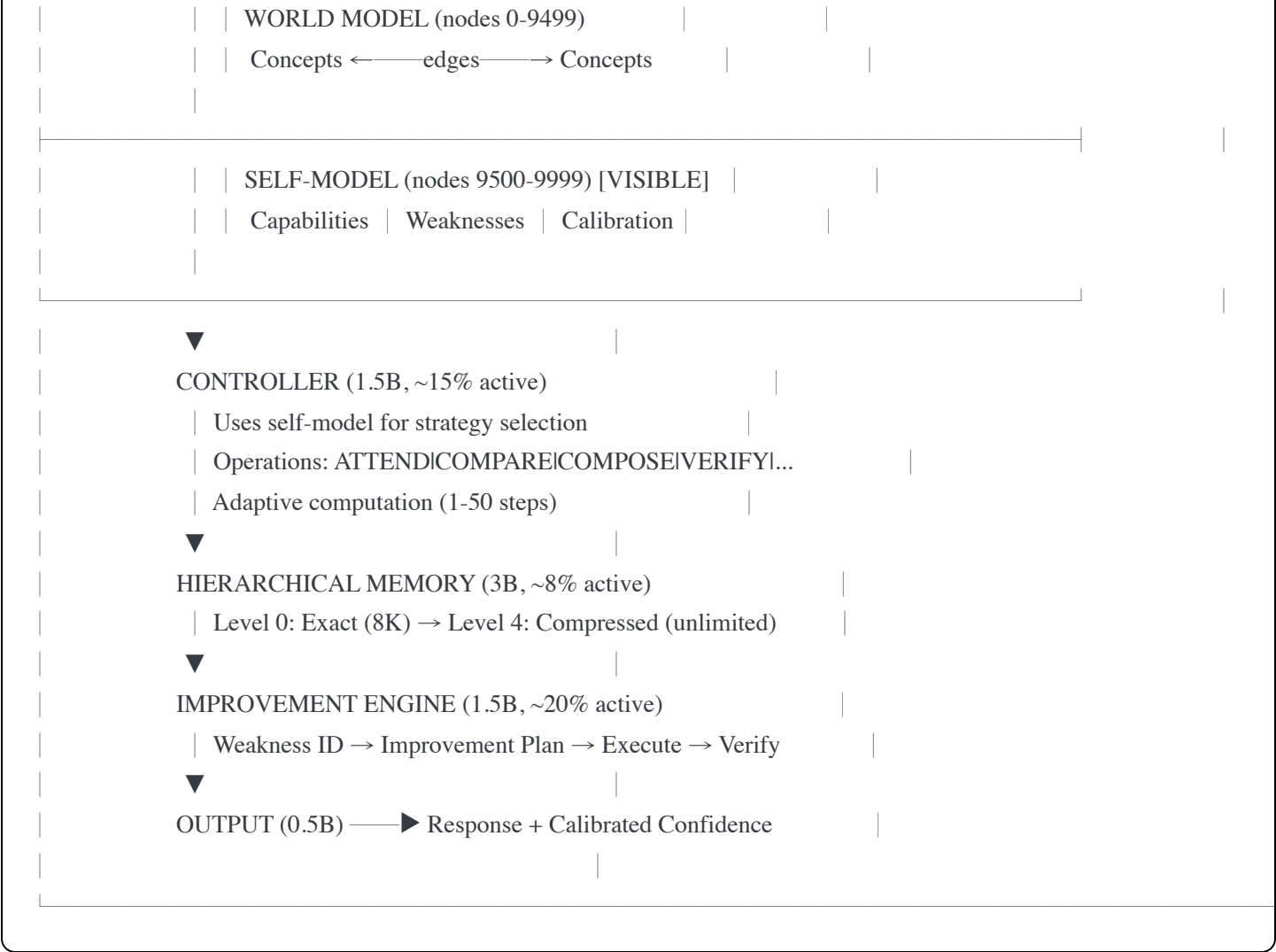
    def transfer_to_workspace(self, transformer, cfa, corpus):
        # Extract concepts via clustering
        concept_bank = []
        for text in corpus:
            hidden = transformer(text).last_hidden_state
            clusters = kmeans(hidden.reshape(-1, hidden.size(-1)), k=1000)
            concept_bank.extend(clusters.centers)

        cfa.workspace.node_embeddings.data[:len(concept_bank)] = torch.stack(concept_bank)
```

Overall speedup estimate: ~1.5-2× vs training from scratch

Appendix K: Complete Architecture Diagram





- GPU 0: Perceptual + Workspace
- GPU 1: Controller + Memory + Output

Appendix M: Complexity Summary

Operation	Transformer	CFA	Advantage
Full context attention	$O(n^2)$	$O(n)$ local + $O(a^2)$ workspace	10,000× for 100K context
Memory retrieval	$O(n)$ scan	$O(\log m)$ learned index	Scales to millions
Parameter activation	100%	10-15%	7-10× compute reduction
Gradient sync	$O(\text{params})$ dense	$O(\text{active})$ sparse	10× comm reduction
Reasoning depth	Fixed layers	Adaptive (1-50 steps)	Variable compute
Context length	~128K max	Unlimited (hierarchical)	∞ advantage